

Tribhuvan University

Faculty of Humanities and Social Sciences

Masters of Computer Application (MCA)

1st Semester

Advanced Database Management System(MCA-503)

Unit 7: Enhanced Data Models: Introduction to Active, Temporal,
Spatial, Multimedia and Deductive Databases [5 Hrs.]

Instructor

Tekendra Nath Yogi (M.SC CSIT TU)

Tekendranath@gmail.com

Contents

- **Unit 7: Enhanced Data Models: Introduction to Active, Temporal, Spatial, Multimedia and Deductive Databases [5 Hrs.]**
 - 7.1. Active Database Concepts and Triggers;
 - 7.2. Temporal Database Concepts;
 - 7.3. Spatial Database Concepts;
 - 7.4. Multimedia Database Concepts;
 - 7.5. Introduction to Deductive Databases.

7.1 Active Database Concepts and Triggers

- Database systems implement rules that specify actions automatically triggered by certain events
- Triggers
 - Technique for specifying certain types of active rules
- Commercial relational DBMSs have various versions of triggers available
 - Oracle syntax used to illustrate concepts

7.1 Active Database Concepts and Triggers Cont'd...

- **Generalized Model for Active Databases and Oracle Triggers:**
- Event-condition-action (ECA) model
 - Event triggers a rule
 - Usually database update operations
 - Condition determines whether rule action should be completed
 - Optional
 - Action will complete only if condition evaluates to true
 - Action to be taken
 - Sequence of SQL statements, transaction, or external program

7.1 Active Database Concepts and Triggers Cont'd...

- Example:
- Events that may cause a change in value of Total_sal attribute
 - Inserting new employee
 - Changing salary
 - Reassigning or deleting employees

EMPLOYEE

Name	<u>Ssn</u>	Salary	Dno	Supervisor_ssn
------	------------	--------	-----	----------------

DEPARTMENT

Dname	<u>Dno</u>	Total_sal	Manager_ssn
-------	------------	-----------	-------------

Figure : A simplified COMPANY database used for active rule examples

7.1 Active Database Concepts and Triggers Cont'd...

- Condition to be evaluated
 - Check that value of Dno attribute is not NULL
- Action to be taken
 - Automatically update the value of Total_sal

7.1 Active Database Concepts and Triggers Cont'd...

```
(a) R1: CREATE TRIGGER Total_sal1
      AFTER INSERT ON EMPLOYEE
      FOR EACH ROW
      WHEN ( NEW.Dno IS NOT NULL )
          UPDATE DEPARTMENT
          SET Total_sal = Total_sal + NEW.Salary
          WHERE Dno = NEW.Dno;

R2: CREATE TRIGGER Total_sal2
      AFTER UPDATE OF Salary ON EMPLOYEE
      FOR EACH ROW
      WHEN ( NEW.Dno IS NOT NULL )
          UPDATE DEPARTMENT
          SET Total_sal = Total_sal + NEW.Salary - OLD.Salary
          WHERE Dno = NEW.Dno;

R3: CREATE TRIGGER Total_sal3
      AFTER UPDATE OF Dno ON EMPLOYEE
      FOR EACH ROW
      BEGIN
          UPDATE DEPARTMENT
          SET Total_sal = Total_sal + NEW.Salary
          WHERE Dno = NEW.Dno;
          UPDATE DEPARTMENT
          SET Total_sal = Total_sal - OLD.Salary
          WHERE Dno = OLD.Dno;
      END;
```

Figure Specifying active rules as triggers in Oracle notation (a) Triggers for automatically maintaining the consistency of Total_sal of DEPARTMENT

7.1 Active Database Concepts and Triggers Cont'd...

```
R4: CREATE TRIGGER Total_sal4
      AFTER DELETE ON EMPLOYEE
      FOR EACH ROW
      WHEN ( OLD.Dno IS NOT NULL)
          UPDATE DEPARTMENT
          SET Total_sal = Total_sal - OLD.Salary
          WHERE Dno = OLD.Dno;
```

```
(b) R5: CREATE TRIGGER Inform_supervisor1
          BEFORE INSERT OR UPDATE OF Salary, Supervisor_ssn
          ON EMPLOYEE
          FOR EACH ROW
          WHEN ( NEW.Salary > ( SELECT Salary FROM EMPLOYEE
                                WHERE Ssn = NEW.Supervisor_ssn ) )
              inform_supervisor(NEW.Supervisor_ssn, NEW.Ssn );
```

Figure (cont'd.) Specifying active rules as triggers in Oracle notation (b) Trigger for comparing an employee's salary with that of his or her supervisor

7.1 Active Database Concepts and Triggers Cont'd...

- **Design and Implementation Issues for Active Databases:**
- Deactivated rule
 - Will not be triggered by the triggering event
- Activate command
 - Makes the rule active again
- Drop command
 - Deletes the rule from the system
- Approach: group rules into rule sets
 - Entire rule set can be activated, deactivated, or dropped

7.1 Active Database Concepts and Triggers Cont'd...

- Timing of action
 - Before trigger executes trigger before executing event that caused the trigger
 - After trigger executes trigger after executing the event
 - Instead of trigger executes trigger instead of executing the event
- Action can be considered separate transaction
 - Or part of same transaction that triggered the rule

7.1 Active Database Concepts and Triggers Cont'd...

- Rule consideration
 - Immediate consideration
 - Condition evaluated as part of same transaction
 - Evaluate condition either before, after, or instead of executing the triggering event
 - Deferred consideration
 - Condition evaluated at the end of the transaction
 - Detached consideration
 - Condition evaluated as a separate transaction

7.1 Active Database Concepts and Triggers Cont'd...

- Row-level rule
 - Rule considered separately for each row
- Statement-level rule
 - Rule considered once for entire statement
- Difficult to guarantee consistency and termination of rules

7.1 Active Database Concepts and Triggers Cont'd...

- **Examples of Statement-Level Active Rules in STARBURST:**

```
R1S: CREATE RULE Total_sal1 ON EMPLOYEE
      WHEN   INSERTED
      IF      EXISTS      ( SELECT * FROM INSERTED WHERE Dno IS NOT NULL)
      THEN   UPDATE      DEPARTMENT AS D
            SET           D.Total_sal = D.Total_sal +
                        ( SELECT SUM (I.Salary) FROM INSERTED AS I WHERE D.Dno = I.Dno )
            WHERE        D.Dno IN ( SELECT Dno FROM INSERTED );
```

```
R2S: CREATE RULE Total_sal2 ON EMPLOYEE
      WHEN   UPDATED      ( Salary )
      IF      EXISTS      ( SELECT * FROM NEW-UPDATED WHERE Dno IS NOT NULL)
            OR EXISTS      ( SELECT * FROM OLD-UPDATED WHERE Dno IS NOT NULL)
      THEN   UPDATE      DEPARTMENT AS D
            SET           D.Total_sal = D.Total_sal +
                        ( SELECT SUM (N.Salary) FROM NEW-UPDATED AS N
                          WHERE D.Dno = N.Dno ) -
                        ( SELECT SUM (O.Salary) FROM OLD-UPDATED AS O
                          WHERE D.Dno = O.Dno )
            WHERE        D.Dno IN ( SELECT Dno FROM NEW-UPDATED ) OR
                        D.Dno IN ( SELECT Dno FROM OLD-UPDATED);
```

Figure :(continues) Active rules using statement-level semantics in STARBURST notation

7.1 Active Database Concepts and Triggers Cont'd...

```
R3S: CREATE RULE Total_sal3 ON EMPLOYEE
      WHEN   UPDATED   ( Dno )
      THEN   UPDATE    DEPARTMENT AS D
            SET        D.Total_sal = D.Total_sal +
                      ( SELECT SUM (N.Salary) FROM NEW-UPDATED AS N
                        WHERE D.Dno = N.Dno )
      WHERE   D.Dno IN ( SELECT Dno FROM NEW-UPDATED );
      UPDATE  DEPARTMENT AS D
      SET     D.Total_sal = Total_sal -
            ( SELECT SUM (O.Salary) FROM OLD-UPDATED AS O
              WHERE D.Dno = O.Dno )
      WHERE   D.Dno IN ( SELECT Dno FROM OLD-UPDATED );
```

Figure : (cont'd.) Active rules using statement-level semantics in STARBURST notation

7.1 Active Database Concepts and Triggers Cont'd...

- **Potential Applications for Active Databases:**

- Allow notification of certain conditions that occur
- Enforce integrity constraints
- Automatically maintain derived data
- Maintain consistency of materialized views
- Enable consistency of replicated tables

7.1 Active Database Concepts and Triggers Cont'd...

- **Triggers in SQL-99:**

```
T1: CREATE TRIGGER Total_sal1
    AFTER UPDATE OF Salary ON EMPLOYEE
    REFERENCING OLD ROW AS O, NEW ROW AS N
    FOR EACH ROW
    WHEN ( N.Dno IS NOT NULL )
    UPDATE DEPARTMENT
    SET Total_sal = Total_sal + N.salary - O.salary
    WHERE Dno = N.Dno;

T2: CREATE TRIGGER Total_sal2
    AFTER UPDATE OF Salary ON EMPLOYEE
    REFERENCING OLD TABLE AS O, NEW TABLE AS N
    FOR EACH STATEMENT
    WHEN EXISTS ( SELECT *FROM N WHERE N.Dno IS NOT NULL ) OR
        EXISTS ( SELECT * FROM O WHERE O.Dno IS NOT NULL )
    UPDATE DEPARTMENT AS D
    SET D.Total_sal = D.Total_sal
    + ( SELECT SUM (N.Salary) FROM N WHERE D.Dno=N.Dno )
    - ( SELECT SUM (O.Salary) FROM O WHERE D.Dno=O.Dno )
    WHERE Dno IN ( ( SELECT Dno FROM N ) UNION ( SELECT Dno FROM O ) );
```

Figure Trigger T1 illustrating the syntax for defining triggers in SQL-99

7.2 Temporal Database Concepts

- Temporal databases require some aspect of time when organizing information
 - Healthcare
 - Insurance
 - Reservation systems
 - Scientific databases
- Time considered as ordered sequence of points
 - Granularity determined by the application

7.2 Temporal Database Concepts Cont'd...

- Chronon
 - Term used to describe minimal granularity of a particular application
- Reference point for measuring specific time events
 - Various calendars
- SQL2 temporal data types
 - DATE, TIME, TIMESTAMP, INTERVAL, PERIOD

7.2 Temporal Database Concepts Cont'd...

- Point events or facts
 - Typically associated with a single time point
 - Time series data
- Duration events or facts
 - Associated with specific time period
 - Time period represented by start and end points
- Valid time
 - True in the real world

7.2 Temporal Database Concepts Cont'd...

- Transaction time
 - Value of the system clock when information is valid in the system
- User-defined time
- Bitemporal database
 - Uses valid time and transaction time
- Valid time relations
 - Used to represent history of changes

7.2 Temporal Database Concepts Cont'd...

(a) EMP_VT

Name	<u>Ssn</u>	Salary	Dno	Supervisor_ssn	<u>Vst</u>	Vet
------	------------	--------	-----	----------------	------------	-----

DEPT_VT

Dname	<u>Dno</u>	Total_sal	Manager_ssn	<u>Vst</u>	Vet
-------	------------	-----------	-------------	------------	-----

(b) EMP_TT

Name	<u>Ssn</u>	Salary	Dno	Supervisor_ssn	<u>Tst</u>	Tet
------	------------	--------	-----	----------------	------------	-----

DEPT_TT

Dname	<u>Dno</u>	Total_sal	Manager_ssn	<u>Tst</u>	Tet
-------	------------	-----------	-------------	------------	-----

(c) EMP_BT

Name	<u>Ssn</u>	Salary	Dno	Supervisor_ssn	<u>Vst</u>	Vet	<u>Tst</u>	Tet
------	------------	--------	-----	----------------	------------	-----	------------	-----

DEPT_BT

Dname	<u>Dno</u>	Total_sal	Manager_ssn	<u>Vst</u>	Vet	<u>Tst</u>	Tet
-------	------------	-----------	-------------	------------	-----	------------	-----

Figure: Different types of temporal relational databases (a) Valid time database schema (b) Transaction time database schema (c) Bitemporal database schema

7.2 Temporal Database Concepts Cont'd...

EMP_VT

Name	<u>Ssn</u>	Salary	Dno	Supervisor_ssn	<u>Vst</u>	Vet
Smith	123456789	25000	5	333445555	2002-06-15	2003-05-31
Smith	123456789	30000	5	333445555	2003-06-01	Now
Wong	333445555	25000	4	999887777	1999-08-20	2001-01-31
Wong	333445555	30000	5	999887777	2001-02-01	2002-03-31
Wong	333445555	40000	5	888665555	2002-04-01	Now
Brown	222447777	28000	4	999887777	2001-05-01	2002-08-10
Narayan	666884444	38000	5	333445555	2003-08-01	Now

...

DEPT_VT

Dname	<u>Dno</u>	Manager_ssn	<u>Vst</u>	Vet
Research	5	888665555	2001-09-20	2002-03-31
Research	5	333445555	2002-04-01	Now

...

Figure: Some tuple versions in the valid time relations EMP_VT and DEPT_VT

7.2 Temporal Database Concepts Cont'd...

- Types of updates
 - Proactive
 - Retroactive
 - Simultaneous
- Timestamp recorded whenever change is applied to database
- Bitemporal relations
 - Application requires both valid time and transaction time

7.2 Temporal Database Concepts Cont'd...

- Implementation considerations
 - Store all tuples in the same table
 - Create two tables: one for currently valid information and one for the rest
 - Vertically partition temporal relation attributes into separate relations
 - New tuple created whenever any attribute updated
- Append-only database
 - Keeps complete record of changes and corrections

7.2 Temporal Database Concepts Cont'd...

- Attribute versioning
 - Simple complex object used to store all temporal changes of the object
 - Time-varying attribute
 - Values versioned over time by adding temporal periods to the attribute
 - Non-time-varying attribute
 - Values do not change over time

7.2 Temporal Database Concepts Cont'd...

```
class TEMPORAL_SALARY
{   attribute    Date                Valid_start_time;
    attribute    Date                Valid_end_time;
    attribute    float               Salary;
};

class TEMPORAL_DEPT
{   attribute    Date                Valid_start_time;
    attribute    Date                Valid_end_time;
    attribute    DEPARTMENT_VT      Dept;
};

class TEMPORAL_SUPERVISOR
{   attribute    Date                Valid_start_time;
    attribute    Date                Valid_end_time;
    attribute    EMPLOYEE_VT        Supervisor;
};

class TEMPORAL_LIFESPAN
{   attribute    Date                Valid_start time;
    attribute    Date                Valid end time;
};

class EMPLOYEE_VT
(   extent EMPLOYEES )
{   attribute    list<TEMPORAL_LIFESPAN>    lifespan;
    attribute    string                    Name;
    attribute    string                    Ssn;
    attribute    list<TEMPORAL_SALARY>      Sal_history;
    attribute    list<TEMPORAL_DEPT>        Dept_history;
    attribute    list <TEMPORAL_SUPERVISOR> Supervisor_history;
};
```

Figure: Possible ODL schema for a temporal valid time EMPLOYEE_VT object class using attribute versioning

7.2 Temporal Database Concepts Cont'd...

- TSQL2 language
 - Extends SQL for querying valid time and transaction time tables
 - Used to specify whether a relation is temporal or non-temporal
- Temporal database query conditions may involve time and attributes
 - Pure time condition involves only time
 - Attribute and time conditions

7.2 Temporal Database Concepts Cont'd...

- CREATE TABLE statement
 - Extended with optional AS clause
 - Allows users to declare different temporal options
 - Examples:
 - AS VALID STATE<GRANULARITY> (valid time relation with valid time period)
 - AS TRANSACTION (transaction time relation with transaction time period)
- Keywords STATE and EVENT
 - Specify whether a time period or point is associated with valid time dimension

7.2 Temporal Database Concepts Cont'd...

- Time series data
 - Often used in financial, sales, and economics applications
 - Special type of valid event data
 - Event's time points predetermined according to fixed calendar
 - Managed using specialized time series management systems
 - Supported by some commercial DBMS packages

7.3 Spatial Database Concepts

- Spatial databases support information about objects in multidimensional space
 - Examples: cartographic databases, geographic information systems, weather information databases
- Spatial relationships among the objects are important
- Optimized to query data such as points, lines, and polygons
 - Spatial queries

7.3 Spatial Database Concepts Cont'd...

- Measurement operations
 - Used to measure global properties of single objects
- Spatial analysis operations
 - Uncover spatial relationships within and among mapped data layers
- Flow analysis operations
 - Help determine shortest path between two points

7.3 Spatial Database Concepts Cont'd...

- Location analysis
 - Determine whether given set of points and lines lie within a given polygon
- Digital terrain analysis
 - Used to build three-dimensional models

7.3 Spatial Database Concepts Cont'd...

Analysis Type	Type of operations and Measurements
Measurements	Distance, Perimeter, shape, adjacency and direction
Spatial analysis/statistics	Pattern, autocorrelation, and indexes of similarity and topology using spatial and non spatial data
Flow analysis	Connectivity and shortest path
Location analysis	Analysis of points and lines within a polygon
Terrain analysis	Analysis of points and lines within a polygon
Search	Thematic search, search by region

Table : Common types of analysis for spatial data

7.3 Spatial Database Concepts Cont'd...

- Spatial data types
 - Map data
 - Geographic or spatial features of objects in a map
 - Attribute data
 - Descriptive data associated with map features
 - Image data
 - Satellite images
- Models of spatial information
 - Field models
 - Object models

7.3 Spatial Database Concepts Cont'd...

- Spatial operator categories
 - Topological operators
 - Properties do not change when topological transformations applied
 - Projective operators
 - Express concavity/convexity of objects
 - Metric operators
 - Specifically describe object's geometry
 - Dynamic spatial operators
 - Create, destroy, and update

7.3 Spatial Database Concepts Cont'd...

- Spatial queries
 - Range queries
 - Example: find all hospitals within the Metropolitan Atlanta city area
 - Nearest neighbor queries
 - Example: find police car nearest location of a crime
 - Spatial joins or overlays
 - Example: find all homes within two miles of a lake

7.3 Spatial Database Concepts Cont'd...

- Spatial data indexing
 - Grid files
 - R-trees
 - Spatial join index
- Spatial data mining techniques
 - Spatial classification
 - Spatial association
 - Spatial clustering

7.4 Multimedia Database Concepts

- Multimedia databases allow users to store and query images, video, audio, and documents
- Content-based retrieval
 - Automatic analysis
 - Manual identification
 - Color often used in content-based image retrieval
 - Texture and shape
- Object recognition
 - Scale-invariant feature transform (SIFT) approach

7.4 Multimedia Database Concepts Cont'd...

- Semantic tagging of images
 - User-supplied tags
 - Automated generation of image tags
 - Web Ontology Language (OWL) provides concept hierarchy
- Analysis of audio data sources
 - Text-based indexing
 - Content-based indexing

7.5 Introduction to Deductive Databases

- Deductive database uses facts and rules
 - Inference engine can deduce new facts using rules
- Prolog/Datalog notation
 - Based on providing predicates with unique names
 - Predicate has an implicit meaning and a fixed number of arguments
 - If arguments are all constant values, predicate states that a certain fact is true
 - If arguments are variables, considered as a query or part of a rule or constraint

7.5 Introduction to Deductive Databases Cont'd...

- Prolog Notation and The Supervisory Tree

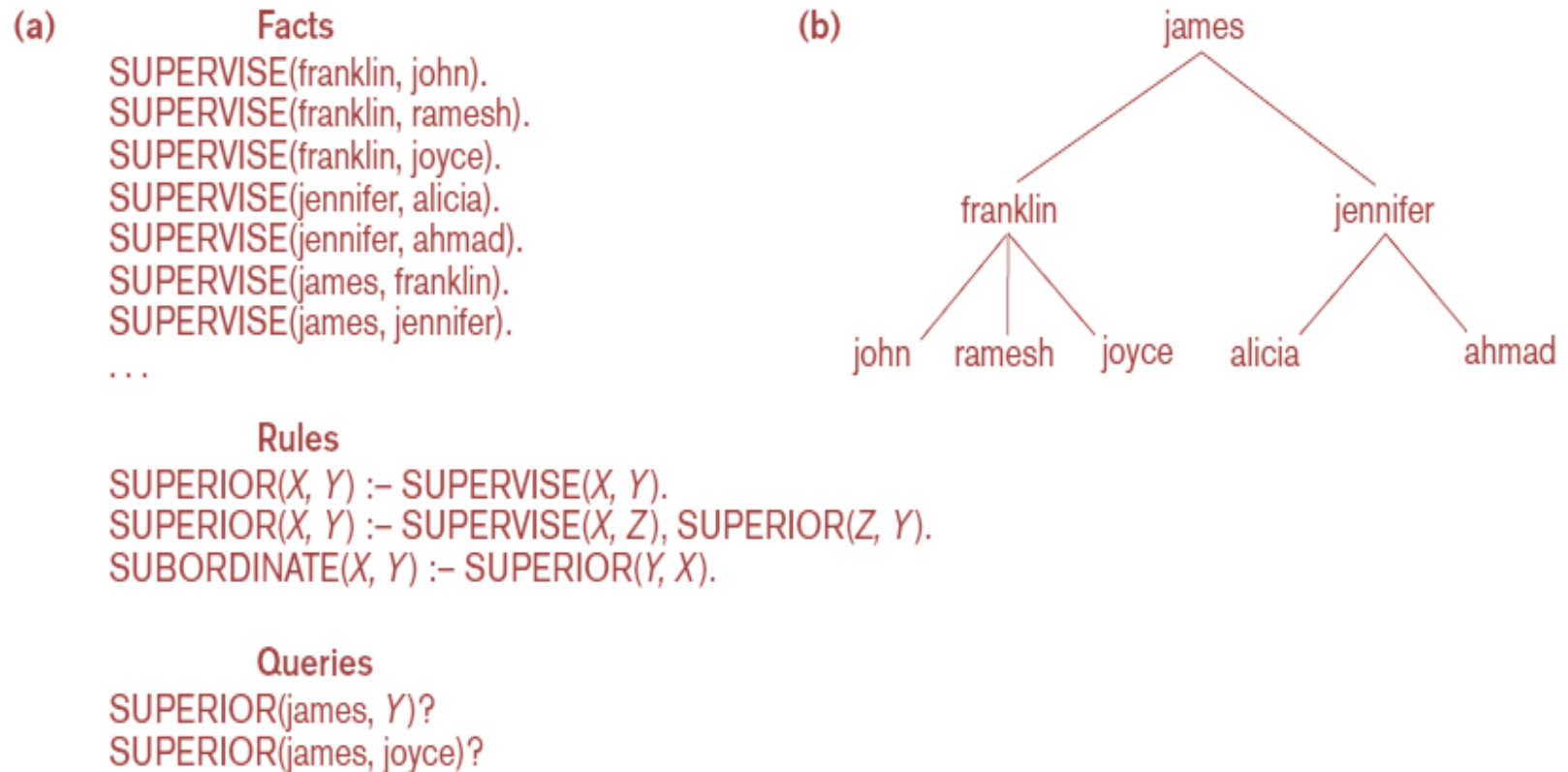


Figure: (a) Prolog notation (b) The supervisory tree

7.5 Introduction to Deductive Databases Cont'd...

- Datalog notation
 - Program built from basic objects called atomic formulas
 - Literals of the form $p(a_1, a_2, \dots, a_n)$
 - p is the predicate name
 - n is the number of arguments for predicate p
- Interpretations of rules
 - Proof-theoretic versus model-theoretic
 - Deductive axioms
 - Ground axioms

7.5 Introduction to Deductive Databases Cont'd...

- | | |
|---|---------------------------|
| 1. SUPERIOR(X, Y) :- SUPERVISE(X, Y). | (rule 1) |
| 2. SUPERIOR(X, Y) :- SUPERVISE(X, Z), SUPERIOR(Z, Y). | (rule 2) |
| 3. SUPERVISE(jennifer, ahmad). | (ground axiom, given) |
| 4. SUPERVISE(james, jennifer). | (ground axiom, given) |
| 5. SUPERIOR(jennifer, ahmad). | (apply rule 1 on 3) |
| 6. SUPERIOR(james, ahmad). | (apply rule 2 on 4 and 5) |

Figure: Proving a new fact

7.5 Introduction to Deductive Databases Cont'd...

- Safe program or rule
 - Generates a finite set of facts
- Non-recursive query
 - Includes only non-recursive predicates

7.5 Introduction to Deductive Databases Cont'd...

- Use of Relational Operations

REL_ONE(A, B, C).

REL_TWO(D, E, F).

REL_THREE(G, H, I, J).

SELECT_ONE_A_EQ_C(X, Y, Z) :- REL_ONE(C, Y, Z).

SELECT_ONE_B_LESS_5(X, Y, Z) :- REL_ONE(X, Y, Z), Y < 5.

SELECT_ONE_A_EQ_C_AND_B_LESS_5(X, Y, Z) :- REL_ONE(C, Y, Z), Y < 5.

SELECT_ONE_A_EQ_C_OR_B_LESS_5(X, Y, Z) :- REL_ONE(C, Y, Z).

SELECT_ONE_A_EQ_C_OR_B_LESS_5(X, Y, Z) :- REL_ONE(X, Y, Z), Y < 5.

PROJECT_THREE_ON_G_H(W, X) :- REL_THREE(W, X, Y, Z).

UNION_ONE_TWO(X, Y, Z) :- REL_ONE(X, Y, Z).

UNION_ONE_TWO(X, Y, Z) :- REL_TWO(X, Y, Z).

INTERSECT_ONE_TWO(X, Y, Z) :- REL_ONE(X, Y, Z), REL_TWO(X, Y, Z).

DIFFERENCE_TWO_ONE(X, Y, Z) :- _TWO(X, Y, Z) NOT(REL_ONE(X, Y, Z)).

CART PROD _ONE_THREE(T, U, V, W, X, Y, Z) :-

REL_ONE(T, U, V), REL_THREE(W, X, Y, Z).

NATURAL_JOIN_ONE_THREE_C_EQ_G(U, V, W, X, Y, Z) :-

REL_ONE(U, V, W), REL_THREE(W, X, Y, Z).

Figure: Predicates for illustrating relational operations

Summary

- Active databases
 - Specify active rules
- Temporal databases
 - Involve time concepts
- Spatial databases
 - Involve spatial characteristics
- Multimedia databases
 - Store images, audio, video, documents, and more
- Deductive databases
 - Prolog and Datalog notation

Thank You !

